

04b

JavaScript – Παραδείγματα

(Τα απολύτως απαραίτητα για Form Validation)



Φώτης Κόκκορας
Πανεπιστήμιο Θεσσαλίας



Επικύρωση Φόρμας στον Client

(Client Side Form Validation) Βασικές Έννοιες

- ❖ Διαδικασία ελέγχου/επικύρωσης στην πλευρά του client (browser) που αποσκοπεί στο να είναι η φόρμα **σωστά συμπληρωμένη**, πριν αποσταλεί (γίνει submit). Υλοποιείται με JavaScript.
 - ❑ **σωστά συμπληρωμένη**: το τι είναι σωστό το καθορίζουν οι ανάγκες της web εφαρμογής που φτιάχνουμε (π.χ. πιθανή database στην οποία θα αποθηκευτούν τα δεδομένα) και στοιχειώδεις κανόνες ασφάλειας.
 - π.χ. ένα πεδίο κειμένου για καταχώρηση ηλικίας πρέπει να έχει σίγουρα αριθμό μέσα, αλλά το αποδεκτό εύρος τιμών καθορίζεται από την εκάστοτε εφαρμογή!
- ❖ Η σύνδεση της φόρμας με τον validator γίνεται στον HTML κώδικα της φόρμας, π.χ.:
`<form name="f1" id="f1" method="post" action="foo.php" onsubmit="return validate_form();">`
 - ❑ η συνάρτηση μπορεί να έχει ότι όνομα θέλουμε αλλά πρέπει να επιστρέφει true ή false
 - true: η υποβολή της φόρμας θα γίνει
 - false: η υποβολή της φόρμας ακυρώνεται
- ❖ Στην απλούστερη εκδοχή, ο validator ελέγχει ένα-ένα τα στοιχεία της φόρμας και ανάλογα κάνει επιτρεπτή την υποβολή ή όχι της φόρμας, βγάζοντας και κατάλληλα μηνύματα στον χρήστη.
 - ❑ τεχνικά μας ενδιαφέρει το πώς βλέπει/διαβάζει η JavaScript τα στοιχεία της φόρμας!

Διάβασμα Φόρμας με JavaScript (1/2)

(με βάση το id του στοιχείου διεπαφής)

- ❖ Μια HTML φόρμα μπορεί να έχει τριών ειδών στοιχεία διεπαφής:
 - ❑ στοιχεία που συμπληρώνονται: α) **input** τύπου *text* ή *password*, β) **textarea**
 - ❑ στοιχεία που επιλέγονται: **select** (λίστες ανοιχτές (listbox) ή πτυσσόμενες (combobox))
 - ❑ στοιχεία που "clickάρονται": **input** τύπου *radio* (radiobuttons) ή *checkbox* (checkboxes)
- ❖ Διάβασμα "περιεχομένου" **με βάση το id** του HTML στοιχείου (έστω id="myid"):
 - ❑ στοιχεία που συμπληρώνονται (διαβάζουμε το καταχωρημένο κείμενο):
 - **var x = document.getElementById("myid").value;**
 - ❑ στοιχεία που επιλέγονται (διαβάζουμε τον δείκτη της επιλογής – η πρώτη επιλογή έχει 0):
 - **var x = document.getElementById("myid").selectedIndex;**
 - **προσοχή:** άλλο ο "δείκτης" και άλλο το "value" που στέλνει το στοιχείο κατά την υποβολή!
 - ❑ στοιχεία που "clickάρονται" (διαβάζουμε την τιμή του checked: **true** στα checkαρισμένα, **false** στα μη-checkαρισμένα)
 - **var x = document.getElementById("myid").checked;**

Διάβασμα Φόρμας με JavaScript (2/2)

(με βάση το name του στοιχείου διεπαφής – **παρωχημένος τρόπος**)

- ❖ Διάβασμα "περιεχομένου" **με βάση το name** του HTML στοιχείου (έστω name="foo")
 - ❑ όπως πριν αλλά αντί της getElementById() χρησιμοποιούμε την **getElementByName()**
 - ❑ στοιχεία που συμπληρώνονται (διαβάζουμε το καταχωρημένο κείμενο):
 - **var x = document.getElementById("foo").value;**
 - ❑ στοιχεία που επιλέγονται (διαβάζουμε τον δείκτη της επιλογής – η πρώτη επιλογή έχει 0):
 - **var x = document.getElementById("foo").selectedIndex;**
 - **προσοχή:** άλλο ο "δείκτης" και άλλο το "value" που στέλνει το στοιχείο κατά την υποβολή!
 - ❑ στοιχεία που "clickάρονται" (διαβάζουμε την τιμή του checked: **true** στα checkαρισμένα, **false** στα μη-checkαρισμένα)
 - **var x = document.getElementById("foo").checked;**
- ❖ Διάβασμα "περιεχομένου" **με βάση τον πίνακα φορμών** του DOM (παλιός τρόπος):
 - ❑ όπως πριν αλλά αντί της getElementById() βάζουμε **forms["form1"]["elementName"]**
 - **form1** είναι το name της φόρμας (εδώ είναι παράδειγμα)
 - **elementName** είναι το name του στοιχείου διεπαφής (εδώ είναι παράδειγμα)
 - ❑ Τα προηγούμενα 3 παραδείγματα σε αυτή την εκδοχή θα είναι αντίστοιχα:
 - **var x = document.forms["form1"]["foo"].value;**
 - **var x = document.forms["form1"]["foo"].selectedIndex;**
 - **var x = document.forms["form1"]["foo"].checked;**

Χρήσιμα σε JavaScript

❖ Αλφαριθμητικά:

- ❑ ιδιότητα **length**: μήκος αλφαριθμητικού
 - `var x = mystring.length;` //το x θα έχει το μήκος του mystring
- ❑ μέθοδος **indexOf(searchstring)**: θέση μέσα στο αλφαριθμητικό στην οποία βρίσκεται το searchstring.
 - `var x = mystring.indexOf("@");` //το x θα έχει τη θέση στο mystring στην οποία υπάρχει το @
 - [παράδειγμα στο w3schools](#)
- ❑ μέθοδος **concat(str1, str1, ..., strN)**: κολλάει στο αλφαριθμητικό τα str1, str2, κτλ.
 - `var bigString = mystring.concat("hello", "hi");` //κολλάει στο mystring τα "hello" και "hi"
 - [παράδειγμα στο w3schools](#)
- ❑ μέθοδος **split(separator, limit)**: τεμαχίζει το αλφαριθμητικό με βάση τον separator και επιστρέφει τα πρώτα limit σε πλήθος τμήματα.
 - [παράδειγμα στο w3schools](#)
- ❑ [περισσότερα στο w3schools](#)

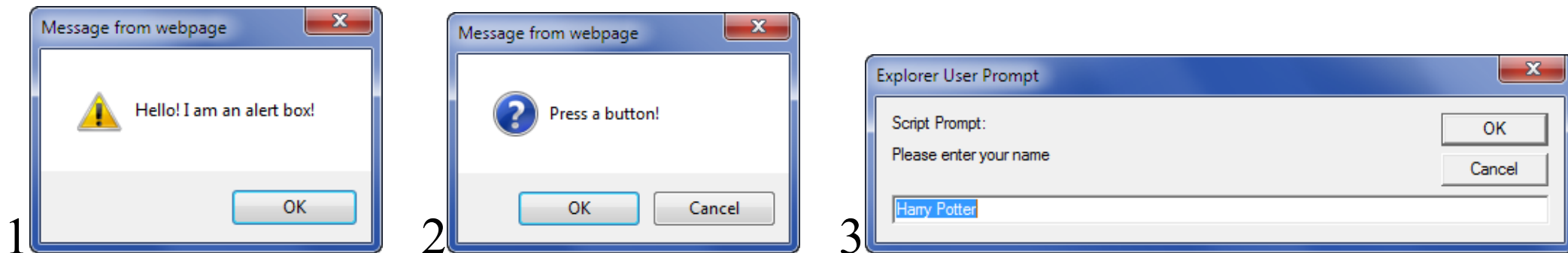
❖ Γενικές Συναρτήσεις

- ❑ **isNaN(value)**: επιστρέφει true αν το value ΔΕΝ είναι αριθμός (**i**s **N**ot **a** **N**umber) διαφορετικά επιστρέφει false.
- ❑ [Δείτε περισσότερες στο w3schools](#)



Παράθυρα Μηνυμάτων

pop-up boxes



1. `alert("Hello! I am an alert box!");`
2. `var r=confirm("Press a button!");`
 - ❑ αν ο χρήστης επιλέξει OK τότε `r=true` ενώ αν επιλέξει Cancel τότε `r=false`
3. `var x=prompt("Please enter your name", "Harry Potter");`
 - ❑ η δεύτερη παράμετρος είναι το προ-συμπληρωμένο κείμενο
 - ❑ επιστρέφει στο `x` το κείμενο που πληκτρολογήθηκε στο textbox
 - ❑ αν πατηθεί το Cancel τότε `x=null`.

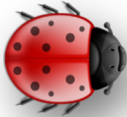
❖ [Δείτε και στο w3schools.](http://w3schools.com)



Regular Expressions

(Κανονικές Εκφράσεις)

- ❖ Μια regular expression είναι ένα αλφαριθμητικό (object) που περιγράφει με γενικό τρόπο ένα pattern χαρακτήρων.
 - ❑ Είναι χρήσιμα στο να περιγράψουμε τι ακριβώς ψάχνουμε μέσα σε ένα αλφαριθμητικό και πιθανώς να προβούμε σε επιπλέον λειτουργίες (πχ αντικατάσταση τμήματος με άλλο).
- ❖ Επίσης, μια regular expression αρχικοποιείται με: `var foo = new RegExp(regex)`
 - ❑ όπου το **regex** είναι το pattern ([δείτε στο w3schools για γλώσσα περιγραφής pattern](#))
 - ❑ **Παράδειγμα:** `var regex = new RegExp(/\\d/);` //όλα τα ψηφία (*digits*), 0 ως 9
 - στο παράδειγμα, το regular expression είναι το **\\d**
- ❖ **Βασικές Μέθοδοι:** έστω η κανονική έκφραση: `r = new RegExp(/tei/i);` δηλαδή το αλφαριθμητικό 'tei' σε case-insensitive εκδοχή (δεν ενδιαφέρει το πεζά-κεφαλαία).
 - ❑ **test()** (δηλαδή έλεγχος ύπαρξης του pattern μέσα στο 'abcdtEi1234')
 - `x = r.test('abcdtEi1234');`
 - το x θα πάρει την τιμή true αν το **tei** βρεθεί μέσα στο **abcdtEi1234**, αλλιώς x=false
 - ❑ **exec()** (εξαγωγή του τμήματος του **abcdtEi1234** που ταιριάζει με το pattern)
 - `x = r.exec('abcdtEi1234')`
 - το x θα πάρει την τιμή του τμήματος του αλφαριθμητικού 'abcdtEi1234' που ταιριάζει με την κανονική έκφραση **/tei/i**, δηλαδή με το **'tEi'**.



Αποσφαλμάτωση JavaScript

- ❖ Οι browsers έχουν ενσωματωμένο debugger για JavaScript.
- ❖ Για τον JavaScript debugger πατήστε **F12** και μετά επιλέξτε τον **Debugger** σε Firefox ή την καρτέλα **Sources** σε Chrome.
 - ☐ Βρείτε τον κώδικα JavaScript που θέλετε να ελέγξετε και **βάλτε breakpoints** κάνοντας click αριστερά, στον αριθμό γραμμής.
 - τα breakpoints (όσα θέλετε) μπαίνουν/βγαίνουν με αυτό τον τρόπο
 - ☐ Μετά πηγαίνετε στη σελίδα σας και κάνετε ότι χρειάζεται για να ενεργοποιηθεί το JavaScript που θέλετε να ελέγξετε. Μόλις η εκτέλεση φτάσει σε breakpoint θα σταματήσει και θα οδηγηθείτε αυτόματα στον debugger.
 - ☐ Μπορείτε να ελέγξετε μεταβλητές, να προχωρήσετε την εκτέλεση γραμμή-γραμμή, κτλ.
 - Δείτε τα toolbars στα δεξιά των screenshots που ακολουθούν. Εντοπίστε τα buttons:
 - ✓ για να προχωράτε βήμα-βήμα χωρίς να μπαίνετε στο σώμα τυχόν συναρτήσεων (step over)
 - ✓ για να μπαίνετε στο σώμα τυχόν συναρτήσεων (step into) ή να βγαίνετε
 - ✓ κτλ
 - η επόμενη προς εκτέλεση εντολή επισημαίνεται συνήθως με βέλος, αριστερά του κώδικα.
- ❖ Ακολουθούν screenshots από τον JavaScript debugger των **Firefox** και **Chrome**



Firefox v.73 – JavaScript Debugger

The screenshot displays the Firefox v.73 JavaScript Debugger interface. The top panel shows a web browser window with the URL `file:///C:/Users/Fotis/Dropbox/TEILar/Cources/Web-App-Dev/01-Labs/lab04-05/1-ok.html`. The page content includes a warning message in Greek: "ΠΡΟΣΟΧΗ: Οι έλεγχοι που υλοποιούνται είναι απλώς παραδείγματα. Κάθε φόρμα απαιτεί ελέγχους που καθορίζονται από την εκάστοτε εφαρμογή που υλοποιούμε." Below this is a registration form with fields for username, password, e-mail, ΑΦΜ, Εισόδημα (income), Φύλο (gender), Ημ/νία Γέν. (date of birth), Ύψος (height), and Ενδιαφέροντα (interests). The form is flanked by two ladybug illustrations.

The bottom panel shows the JavaScript Debugger. The **Debugger** tab is active, displaying the source code of `1-ok.html`. The code includes a function to validate the form data. A red box highlights the line `d1 = new Date("1910-01-01");` at line 203. The **Breakpoints** panel on the right shows a breakpoint set at line 203: `d1 = new Date("1910-01-01");`. The **Watch expressions** panel is also visible.

Chrome v.80 - JavaScript Debugger

The screenshot displays the Chrome v.80 JavaScript Debugger interface. The top section shows the browser window with a login form titled "Εργαστήριο 04-05". The form includes fields for username, password, e-mail, ΑΦΜ, Εισόδημα, Φύλο, and Ημ/νια Γέν., along with a "Υψος" field. A red ladybug icon is visible on the left side of the browser window.

The bottom section shows the "Sources" tab, which is highlighted with a red box. The "1-ok.html" file is open, and the script is paused at line 203, which is also highlighted with a red box. The code at line 203 is:

```
203 d1 = new Date("1910-01-01"); //ημερομηνία ΑΠΟ
```

The right sidebar shows the "Paused on breakpoint" status, with a red box around the "Watch" section. The "Watch" section displays the following variables and their values:

- foo: <not available>
- d: <not available>
- d2: <not available>

The "Call Stack" section shows the following calls:

- validate_form 1-ok.html:203
- onsubmit 1-ok.html:271

The "Scope" section shows the following variables:

- Local
 - result: false
 - username: ""
 - illegalChars: //W//
 - password: ""